

Information Retrieval 10

Bi-Encoders and Dense Retrieval

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

This lecture is based on the following textbook:

Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

Vocabulary mismatch problem: When queries and documents use different terms, keyword-based retrieval cannot find relevant documents

Document expansion (doc2query): Generate anticipated queries from documents and append them. Processed at indexing time with no additional cost at search time

DeepCT / DeepImpact: Learn contextual importance of terms using BERT, improving term weights in the inverted index

This time we learn an entirely new approach that does not rely on vocabulary: retrieval using **Dense Vectors**

Understand the principles and major methods of **Dense Retrieval**, which converts queries and documents into dense vectors and performs fast retrieval through nearest neighbor search in vector space

First Half: Foundations of Dense Retrieval

Limitations of Cross-encoders and Bi-encoders

Approximate Nearest Neighbor (ANN) Search

Sentence-BERT: Learning Sentence Embeddings

Second Half: Major Dense Retrieval Methods

DPR: Dense Passage Retrieval

ANCE: Dynamic Hard Negative Mining

ColBERT: Late Interaction Model

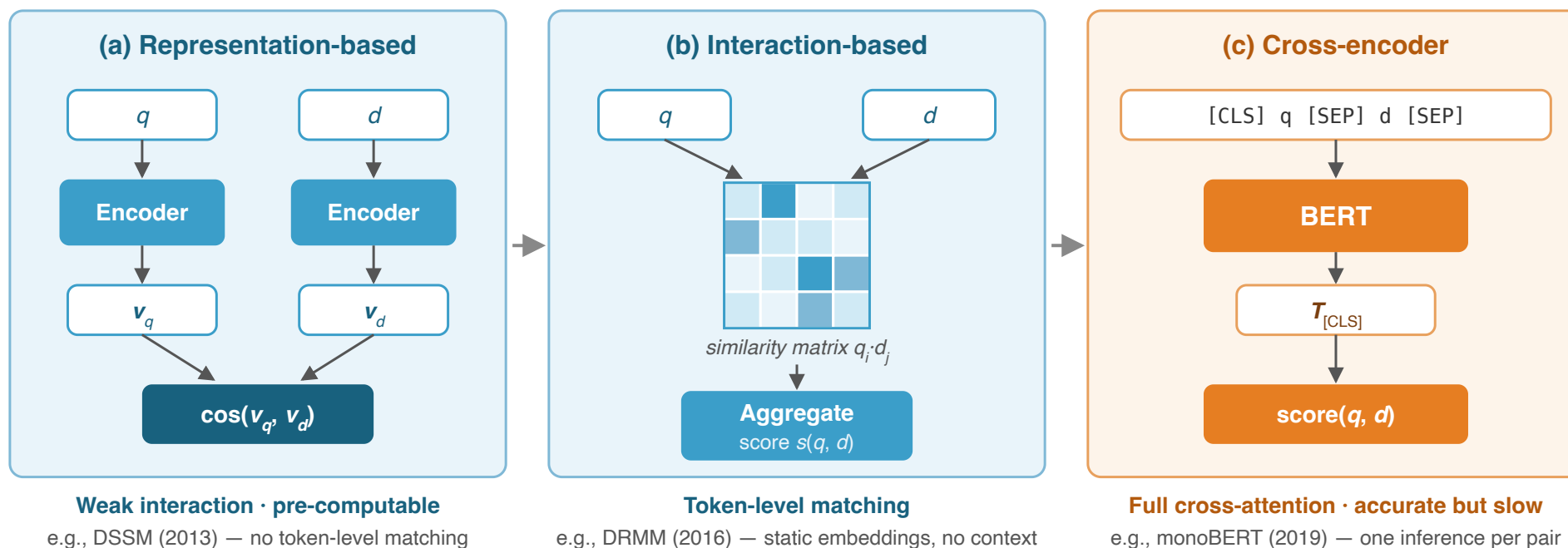
Design Space of Bi-encoders

Limitations of Cross-encoders and Bi-encoders

Why Cross-encoders Cannot Be Used for Retrieval

1. **Representation-based** (DSSM): Independently vectorizes query and document. Weak interaction
2. **Interaction-based** (DRMM): Uses similarity matrices but without context
3. **Cross-encoder** (monoBERT): **Concatenates** query and document as input to BERT. High accuracy but slow

Next step: A model that maintains high accuracy while being **usable for retrieval (first stage)** is needed



Redrawn from Lin et al. (2021), Figure 22. Evolution from representation-based to interaction-based to BERT-based models

Cross-encoders (monoBERT, etc.) concatenate query q and document d as input to BERT:

$$\text{input} = [\text{CLS}] \ q \ [\text{SEP}] \ d \ [\text{SEP}]$$

Problem: BERT must be executed for every query-document pair

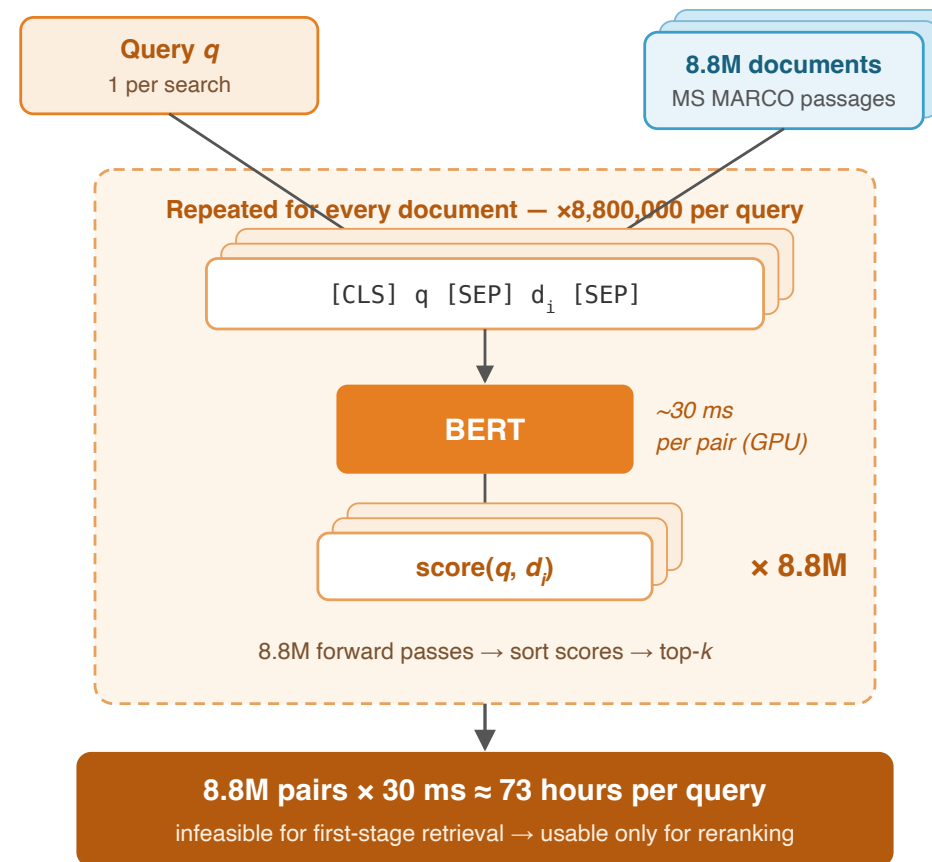
Concrete computational cost:

Collection: 8.8 million documents (MS MARCO)

Inference per pair: ~30ms (GPU)

Per query: 8.8M x 30ms = ~73 hours

-> Cannot be used for first-stage retrieval. Only usable for reranking after narrowing candidates with BM25, etc.



Bi-encoder encodes queries and documents **independently**:

$$\mathbf{q} = E_q(\text{query}), \quad \mathbf{d} = E_d(\text{document})$$

$$\text{score}(q, d) = \text{sim}(\mathbf{q}, \mathbf{d})$$

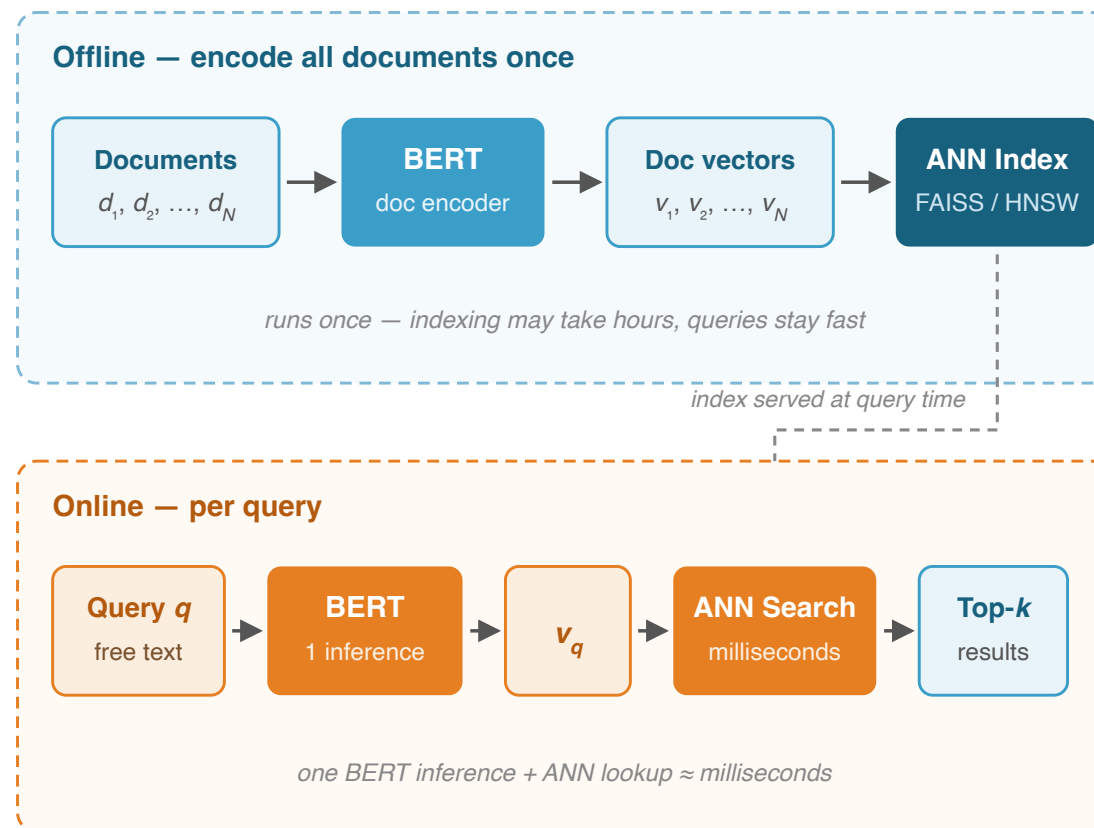
The similarity function `sim` uses **dot product** or **cosine similarity**

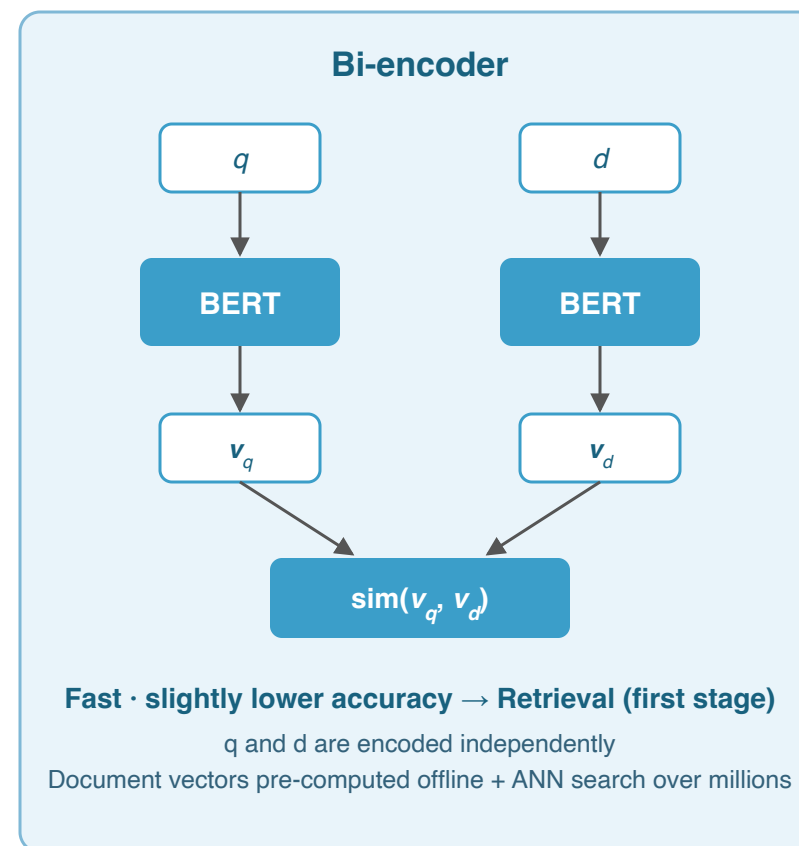
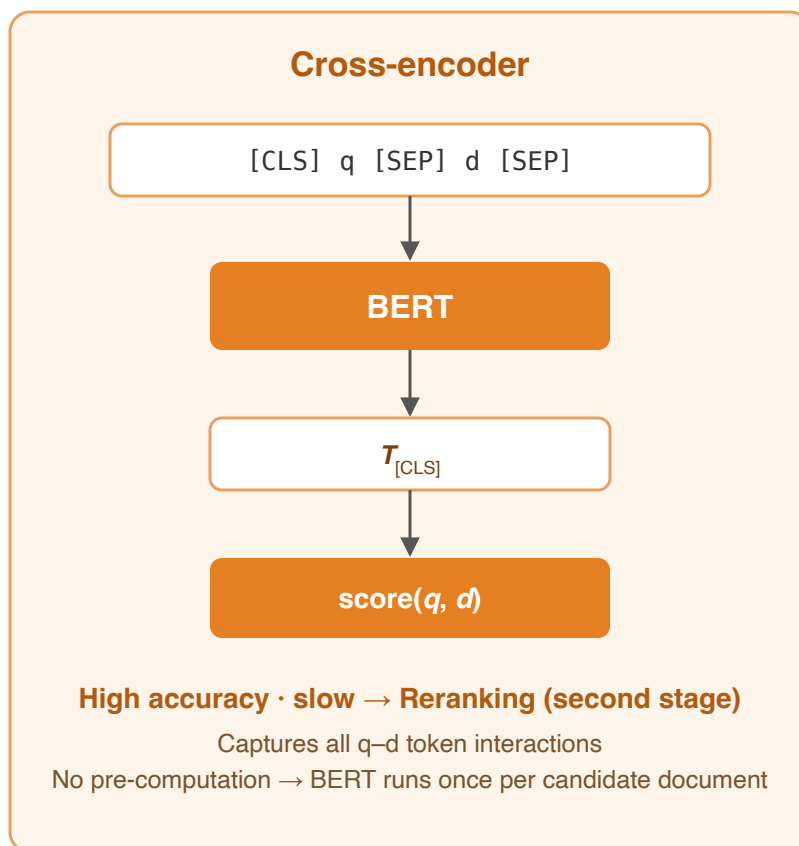
Key advantages:

Pre-compute document vectors: Compute and store vectors for all documents offline

Only encode the query at search time: One BERT inference + vector search

No need for pairwise document comparison ->
Search possible in **milliseconds**





Design philosophy difference: Cross-encoder prioritizes accuracy, Bi-encoder enables pre-computation + fast retrieval. In practice, a multi-stage architecture of **Bi-encoder for retrieval** → **Cross-encoder for reranking** is common

Nearest Neighbor Search

Approximate Nearest Neighbor (ANN) Search

The problem of finding the **most similar document** from dense vectors obtained by a Bi-encoder:

$$d^* = \arg \max_{d \in \mathcal{D}} \text{sim}(\mathbf{q}, \mathbf{d})$$

Exact brute-force search:

Compute similarity with all documents: $O(N \cdot D)$

8.8M documents x 768 dims -> several seconds

Approximate Nearest Neighbor (ANN) search is needed:

No exactness guarantee, but searches in **milliseconds**

In practice, top-document recall can exceed 95%

Exact Search vs Approximate Search

Brute-force

Compute scores with all documents -> **Exact**

Complexity: $O(N \times D)$

8.8M documents: **Several seconds**

ANN (Approximate Nearest Neighbor)

Partition space / build graph to narrow candidates -> **Fast**

Complexity: $O(\log N)$ to $O(N^{0.5})$

8.8M documents: **Milliseconds**

HNSW (Hierarchical Navigable Small World) is an ANN algorithm that uses graph structures

Basic idea:

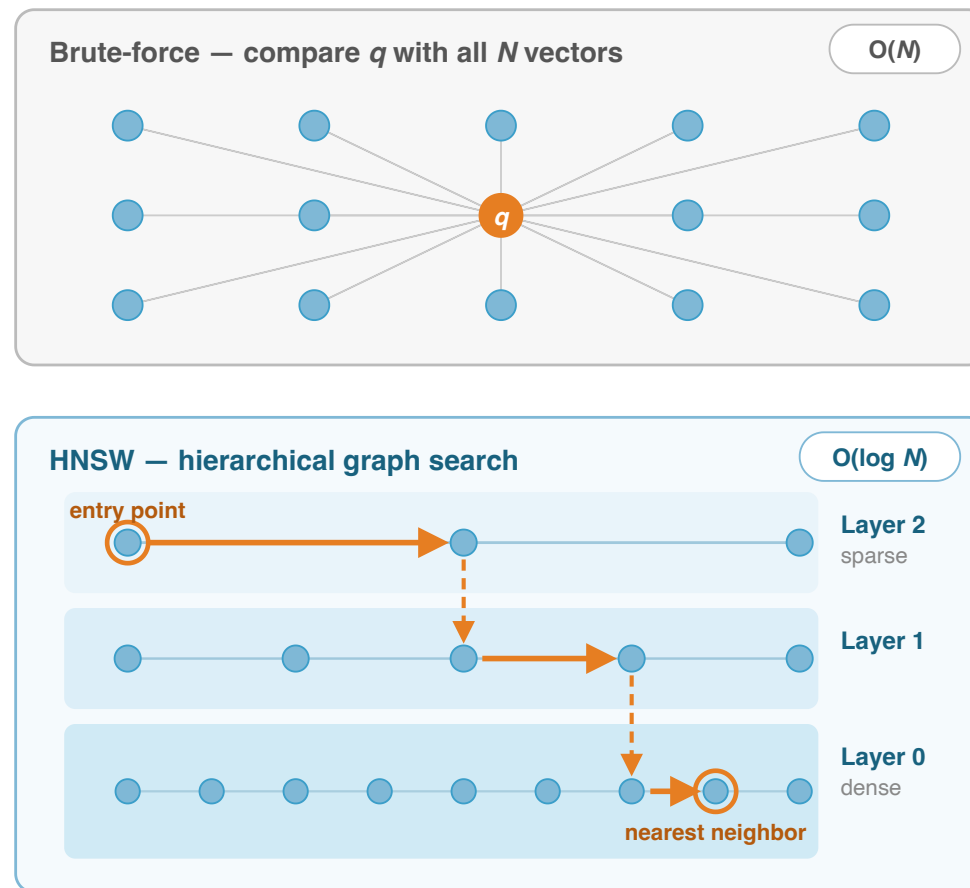
Each vector is a node; edges connect nearby vectors

Hierarchy: Upper layers have coarse, long-distance edges; lower layers are dense

Search flow:

1. Find approximate neighbors in the topmost layer
2. Gradually refine through lower layers
3. Return nearest neighbors at the bottom layer

Comparison with IVF: HNSW uses more memory but is **more accurate**; IVF+PQ is memory-efficient but slightly less accurate



Sentence-BERT

Sentence Embeddings via Siamese Networks

Using BERT directly for sentence similarity computation requires a **Cross-encoder for all pairs**:

Finding the most similar sentence pair among 10,000 sentences

$\binom{10000}{2} \approx 50$ million pairs

~65 hours (V100 GPU)

What about using BERT's [CLS] token directly as a sentence vector?

Experiments showed performance **worse** than GloVe averaged vectors

BERT is trained as a Cross-encoder, and its standalone sentence representation is not optimized

-> A Bi-encoder that can independently produce dense vectors for sentences is needed

Why BERT's [CLS] Is Not Sufficient

Spearman Correlation on STS Benchmark

GloVe average: **58.02**

BERT [CLS] (no fine-tuning): **29.19**

BERT average pooling (no FT): **46.35**

Sentence-BERT: 84.67

Pre-trained BERT's [CLS] does not capture semantic textual similarity. Sentence-BERT explicitly learns sentence representations through a Siamese Network

Siamese Network: Two weight-shared BERTs independently encode two sentences

Structure:

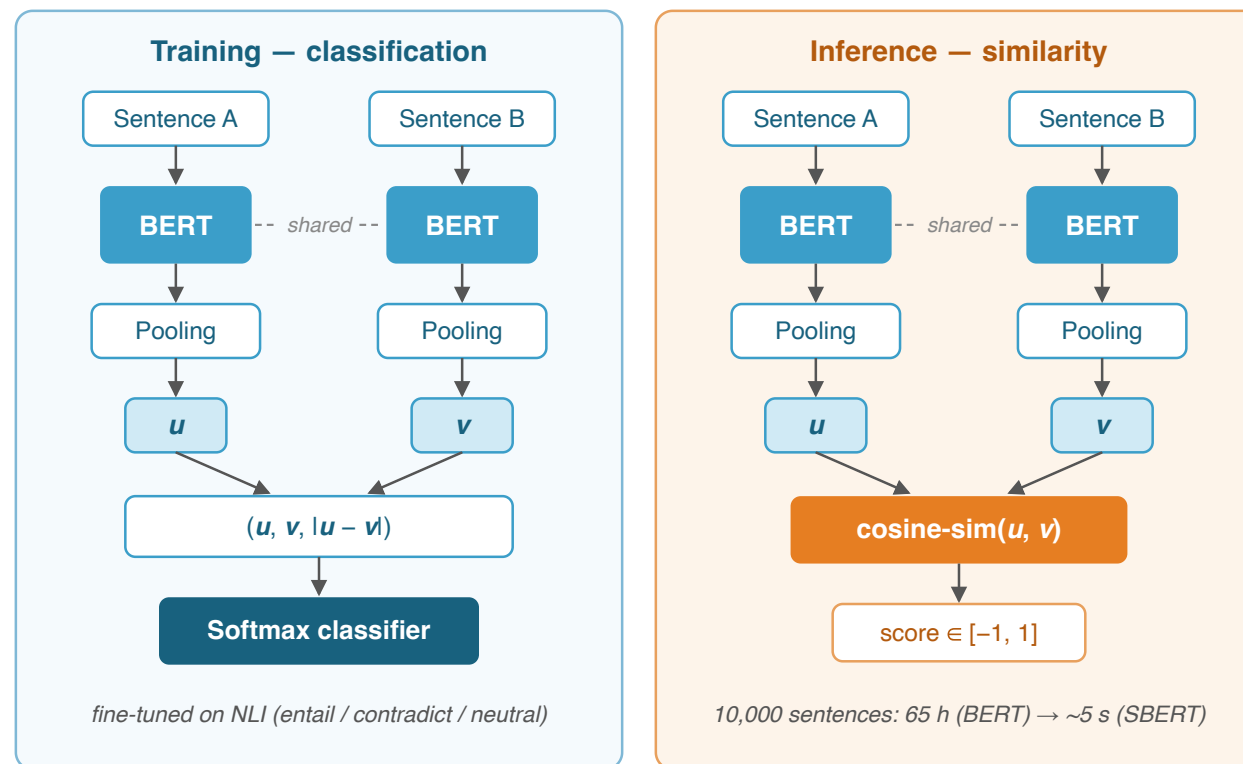
1. Input sentences A and B into separate BERTs
2. **Pool** each BERT's output tokens into sentence vectors $\mathbf{u}$, $\mathbf{v}$
3. Compute the objective function from $\mathbf{u}$, $\mathbf{v}$

Comparison of pooling strategies:

[CLS] token: Lowest performance

Mean pooling: Token average -> **Best**

Max pooling: Per-dimension maximum



Redrawn from Reimers and Gurevych (2019). Two sentences are independently encoded by weight-shared BERTs, and sentence vectors are generated through pooling

Results on STS (Semantic Textual Similarity):

Approaches Cross-encoder BERT accuracy while being **thousands of times faster**

Significantly outperforms GloVe and BERT [CLS]

Inference speed: Comparing 10,000 sentence pairs — BERT: 65 hours -> **SBERT: ~5 seconds**

Significance of Sentence-BERT:

First method to extract **high-quality sentence vectors** from BERT

Foundation technology for sentence similarity search, clustering, and semantic search

Paved the way for Dense Retrieval

Accuracy and Speed Comparison

STS Benchmark (Spearman Correlation)

SBERT-NLI-base	84.67
BERT-base [CLS]	29.19
GloVe average	58.02

DPR — Dense Passage Retrieval

Large-scale Document Retrieval with Bi-encoders

DPR (Dense Passage Retrieval) is a Bi-encoder-based Dense Retrieval model designed for Open-domain QA

Differences from Sentence-BERT:

Sentence-BERT: Learns sentence **similarity**
(NLI/STS data)

DPR: Directly learns to **retrieve relevant documents** for a query

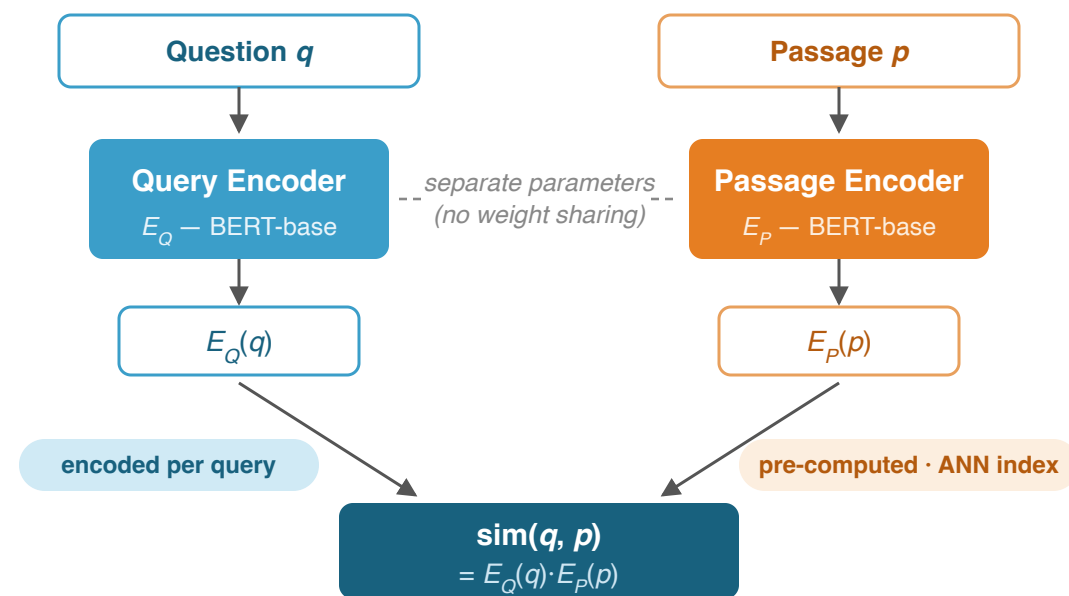
Architecture:

Query encoder E_Q : BERT-base

Document encoder E_P : BERT-base (separate parameters)

[CLS] token from each encoder used as the sentence vector

Similarity: **Dot product** $\text{sim}(q, p) = E_Q(q)^\top E_P(p)$



dot product over 768-d [CLS] vectors — top-k passages via ANN search

The most important technique in DPR training is **in-batch negatives**

Basic idea:

A batch contains B (query, positive document) pairs

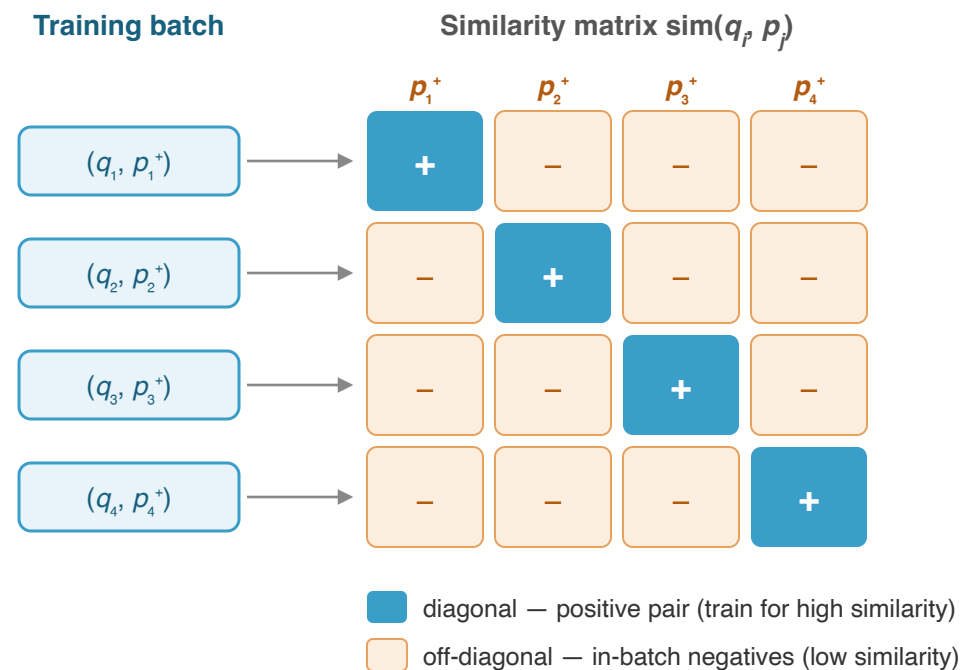
For each query, **positive documents of other queries** are used as negatives

Obtain $B - 1$ **negatives** without additional negative encoding

Loss function (Negative Log-Likelihood):

$$\mathcal{L} = -\log \frac{e^{\text{sim}(q_i, p_i^+)}}{e^{\text{sim}(q_i, p_i^+)} + \sum_{j \neq i} e^{\text{sim}(q_i, p_j^+)}}$$

Larger batch sizes yield more negatives and improve performance



$B = 4 \rightarrow 3$ free negatives per query $\cdot B = 128 \rightarrow 127$ negatives

In-batch negatives alone are insufficient — random negatives are "too easy"

Hard Negatives: Documents that **seem relevant but are actually non-relevant** — e.g., high BM25 score but non-relevant in qrels

Hard Negative strategy in DPR:

Top BM25 results (excluding the correct document) serve as hard negatives

Per query: 1 Gold + 1 BM25 hard negative + in-batch negatives

Effect (Natural Questions, Top-20 Recall):

Random negatives only: 78.4%

Adding BM25 hard negatives: **79.8%**

Hard negatives from another encoder: **up to 82.0%**

Levels of Negative "Difficulty"

Easy Negatives

Randomly sampled documents. Low relevance to the query, easily distinguishable by the model

BM25 Hard Negatives

High BM25 score but non-relevant. Lexically similar but semantically mismatched

DPR Hard Negatives

High DPR score but non-relevant. The examples the model struggles with most

On Natural Questions, DPR's Top-k Recall **significantly surpasses BM25**

Top-20 Recall:

BM25: 59.1%

DPR (single dataset training): **79.4%**

Top-100 Recall:

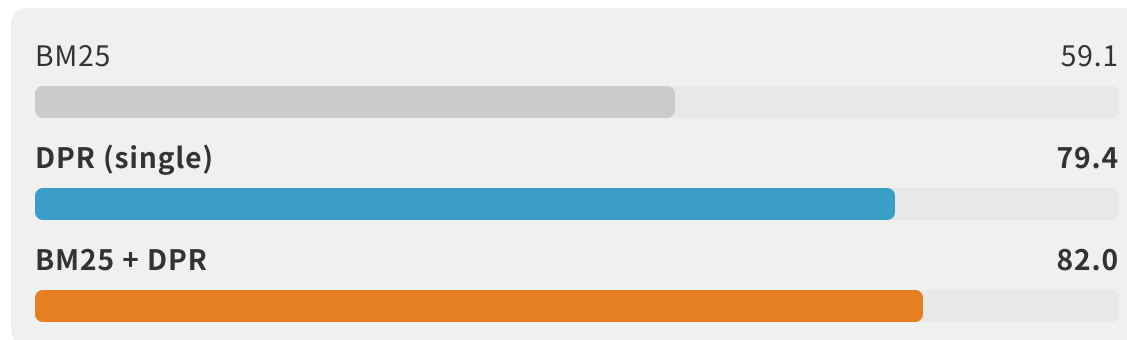
BM25: 73.7% -> DPR: **86.0%**

Combining BM25 + DPR is also effective:

Linear score interpolation achieves **82.0%** (Top-20)

Lexical and semantic matching are complementary

Natural Questions: Top-20 Recall



Source: Karpukhin et al. (2020), Table 2. DPR overcomes vocabulary mismatch and surpasses BM25 by over 20 points

ANCE

Approximate Nearest Neighbor Negative Contrastive Estimation

ANCE (Approximate Nearest Neighbor Negative Contrastive Estimation) obtains hard negatives from **the model's own retrieval results**

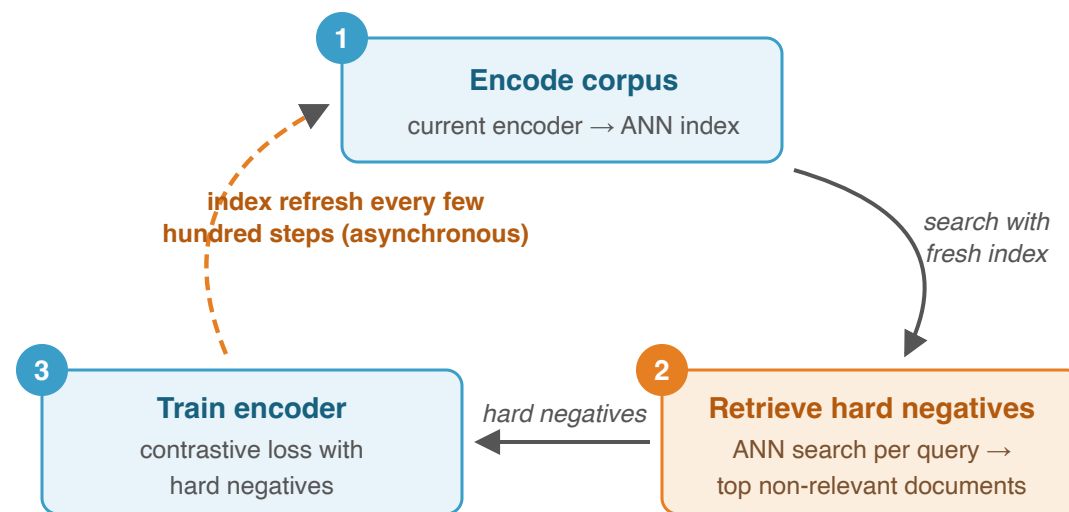
Algorithm:

1. Encode all documents with the current model -> build ANN index
2. ANN search per query -> top non-relevant documents become hard negatives
3. Train several steps on these hard negatives
4. **Periodically** update the index (return to step 1)

Asynchronous updates:

Index reconstruction is costly -> update **every several hundred to thousand steps**

Slightly stale negatives remain effective



negatives go slightly stale between refreshes — still effective, far cheaper

Index update frequency is a trade-off: frequent -> accurate negatives but costly, infrequent -> low cost but stale negatives

ColBERT

Efficient Matching via Late Interaction

Bi-encoders compress an entire document into a **single vector** — inherent limitations:

Information bottleneck:

Diverse content of a long document packed into 768 dimensions

Hard to capture partial matches like "restaurants in Tokyo" vs "tourist spots in Tokyo"

Accuracy gap vs Cross-encoders: Cross-encoders capture **token-level** interactions; Bi-encoders only **sentence-level** similarity

Late Interaction as a solution: Keep **multiple token vectors** per document and match at token level at search time -> approaches Cross-encoder accuracy while retaining pre-computation

Comparison of Representational Power

Bi-encoder: Single Vector

Entire document -> 1 vector

Information is compressed, fine-grained meaning is lost

ColBERT: Token Vector Sets



Each token retains its own vector -> Rich representation

ColBERT (Contextualized Late Interaction over BERT) preserves token-level representations while enabling pre-computation

Encoding:

Query: $\mathbf{Q} = \text{BERT}(q) \in \mathbb{R}^{L_q \times d}$

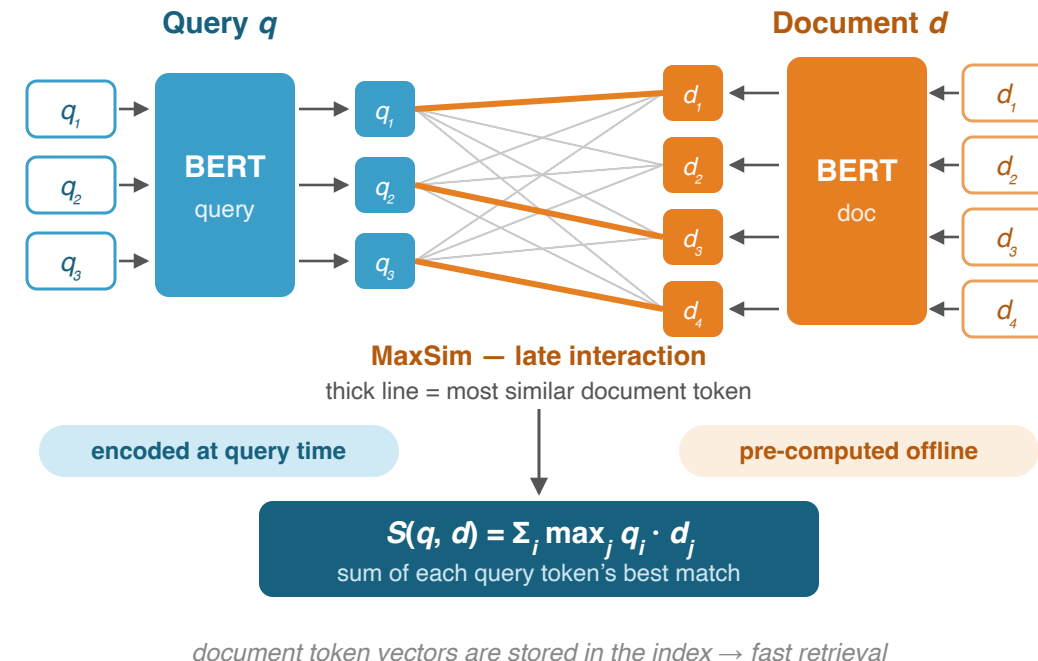
Document: $\mathbf{D} = \text{BERT}(d) \in \mathbb{R}^{L_d \times d}$

Each token output is projected to $d = 128$ dimensions through a linear layer

MaxSim scoring:

$$S(q, d) = \sum_{i=1}^{L_q} \max_{j=1}^{L_d} \mathbf{Q}_i \cdot \mathbf{D}_j$$

Sum the score with the **most similar document token** for each query token



ColBERT's Late Interaction (redrawn from Lin et al. (2021), Figure 27)

Intuitive understanding of MaxSim: Each query token finds the **semantically closest position** in the document

Example: Query "best time for autumn leaves in Kyoto"

"Kyoto" -> high similarity with "Kyoto",

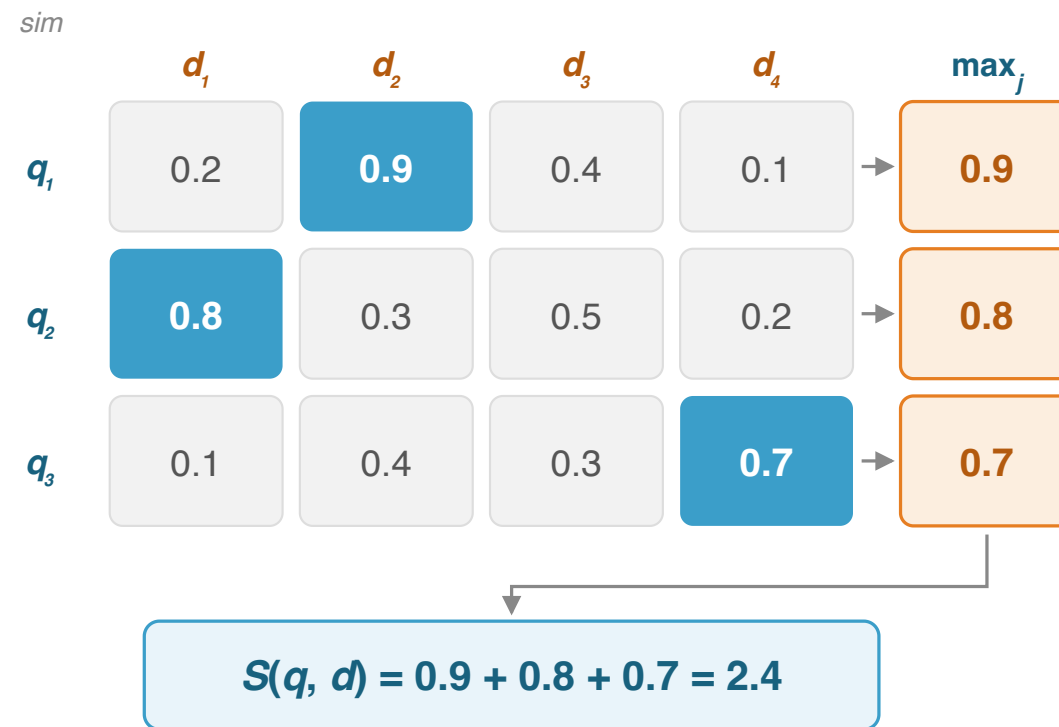
"Kansai", "Arashiyama"

"autumn leaves" -> "foliage", "fall colors"

"best time" -> "season", "November"

vs Cross-encoder: Full attention across all tokens, while ColBERT aggregates per-encoder outputs via MaxSim

vs Bi-encoder: One whole-sentence similarity, while ColBERT sums **per-token maximum similarity** -> Captures partial matches



illustrative values — each query token keeps only its best match

Results on MS MARCO Passage Ranking:

MRR@10: 0.349 (re-ranking), 0.360 (end-to-end)

Accuracy comparable to BERT re-ranker (Cross-encoder)

Tens to hundreds of times faster than exhaustive Cross-encoder comparison

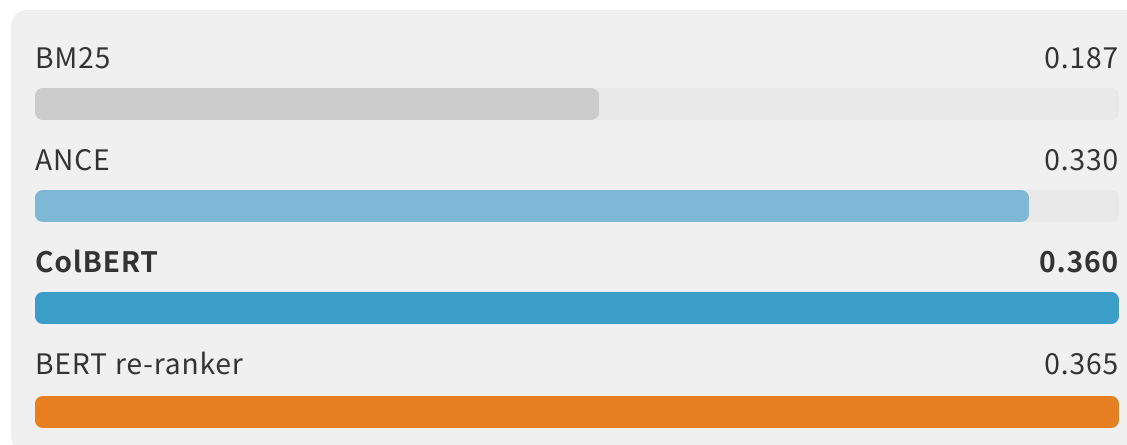
Results on TREC DL 2019:

BM25 + ColBERT re-ranking: **nDCG@10 0.706**, far above BM25 alone (0.506)

Improvements in ColBERTv2 [Santhanam et al., 2022]:

Residual compression (quantized token vectors)
-> **6-10x storage reduction** while maintaining accuracy

MS MARCO: MRR@10 Comparison



Source: Khattab and Zaharia (2020). ColBERT achieves accuracy nearly equivalent to Cross-encoder (BERT re-ranker)

Design Space of Bi-encoders

Method Comparison and Poly-encoder

Method	Interaction	Doc Representation	Negative Strategy	Search Speed	Storage
Cross-encoder	Full Attention	Not pre-computable	BM25 negatives	Very slow	-
Sentence-BERT	None (dot product)	1 vector	NLI/Triplet	Fastest	Small
DPR	None (dot product)	1 vector	In-batch + BM25	Fastest	Small
ANCE	None (dot product)	1 vector	ANN Hard Neg	Fastest	Small
ColBERT	Late (MaxSim)	Token vector sets	Pairwise	Fast	Large
Poly-encoder	Attention codes	1 vector	In-batch	Fast	Small

Key points of the design space: (1) Depth of interaction (None -> Late -> Full), (2) Granularity of document representation (1 vector -> Token vector sets), (3) Quality of negatives (Random -> BM25 -> Model's own). These trade-offs determine the method

1. **Bi-encoder:** Encodes queries and documents independently and retrieves by dense vector similarity. Pre-computation of document vectors enables large-scale retrieval that is impossible with Cross-encoders
2. **DPR / ANCE:** Improve Bi-encoder training through in-batch negatives and ANN-based hard negative mining. Negative quality significantly impacts performance
3. **ColBERT:** A Late Interaction model that retains token-level vectors and aggregates via MaxSim. Achieves intermediate accuracy and speed between Bi-encoders and Cross-encoders
4. **Design space of Dense Retrieval:** Methods are determined by trade-offs in interaction depth, document representation granularity, and negative strategy. In production, hybrid approaches with BM25 are common

Preview of Next Lecture: Learned Sparse Retrieval

SPLADE: Learned sparse representations and vocabulary expansion

Comprehensive comparison of Sparse vs Dense vs Late Interaction

Hybrid search and overall summary